Socket.io

Zero to Hero — Complete Guide

Hinglish Mein — Har Cheez Micro-Step Explanation ke Saath
JavaScript / Node.js / React

Real-time · Bidirectional · Event-driven

Table of Contents

CHAPTER 1

Socket.io Kya Hai?

Pehle Samjho — Real-time Communication Kya Hoti Hai?

Normal websites mein jo bhi data load hota hai, wo tab aata hai jab **tum khud request karte ho** — browser tab refresh karo, ya koi button click karo. Ye purana HTTP ka kaam karne ka tarika hai.

■ *Example: Tum Gmail kholo. Naya email aane pe page khud refresh nahi hota na? Pehle zamaane mein karta tha. Lekin ab nahi karta — kyunki real-time connection hai.*

Real-time communication mein **server khud tumhare paas data push karta hai** — tum request karo ya na karo. Jaise:

- WhatsApp pe message aaya — turant dikhta hai
- Chess app mein opponent ne move kiya — turant dikhta hai
- Uber mein driver ka location live update hota hai
- Online game mein doosre player ka action turant dikhai deta hai

HTTP kaise kaam karta hai (aur problem kya hai)

HTTP ek **request-response** protocol hai. Matlab:

- Client request bhejta hai → Server response deta hai → Connection BAND ho jaata hai

■ *Ye ek phone call jaisa hai jisme aap baat karo, aur turant line cut ho jaaye. Baar baar call karo.*

Iske liye ek jugaad tha jise **Polling** kehte hain — browser har 1-2 second mein server se poochta tha 'kuch naya hai?' — ye bahut wasteful hai.

WebSocket Aaya — Game Changer

WebSocket ek alag protocol hai jo ek baar connection banata hai aur **wo connection permanently khula rehta hai**. Server aur client dono ek doosre ko kabhi bhi data bhej sakte hain bina naya request kiye.

■ *Phone call jaisa — line open hai, dono bol sakte hain kabhi bhi.*

To Socket.io Kya Hai — WebSocket Se Alag Kyun?

Socket.io **WebSocket ke upar ek wrapper library hai**. Matlab WebSocket use karta hai internally, lekin uske upar bohot saari cheezein add karta hai:

Feature	Raw WebSocket	Socket.io
Auto reconnect	Khud banana padega	Built-in hai
Rooms / Groups	Nahi hai	Built-in hai
Namespaces	Nahi hai	Built-in hai
Fallback (agar WS block ho)	Nahi hai	HTTP polling pe fall karta hai
Acknowledgements	Khud banana padega	Built-in hai

Event system	Manually handle karo	Simple .emit() .on()

Socket.io Internally Kaise Kaam Karta Hai?

Jab client connect karta hai, Socket.io ye steps follow karta hai:

- **Step 1:** Pehle normal HTTP request jaati hai — ek 'handshake' hota hai
- **Step 2:** Server aur client dono agree karte hain WebSocket pe upgrade karne ke liye
- **Step 3:** WebSocket connection open ho jaata hai — ab dono ek doosre ko freely data bhej sakte hain
- **Fallback:** Agar WebSocket kisi wajah se kaam na kare (firewall, purana browser), Socket.io automatically HTTP long-polling pe switch kar leta hai

■ *Socho jaise pehle ek WhatsApp call karke poochho 'Kya main aa sakta hoon?' aur phir doosra bole 'Haan aa jao' — tab permanent connection banta hai.*

CHAPTER 2

Installation aur Basic Setup

Step 1 — Node.js Project Initialize Karo

Sabse pehle ek naya folder banao aur npm project shuru karo:

```
mkdir my-socketio-app

cd my-socketio-app

npm init -y

# -y matlab sab questions ka default answer haan — package.json ban jaayega
```

■ *npm init -y se ek package.json file banti hai — ye tumhare project ka ID card hai. Har dependency, script, project name ismein hoti hai.*

Step 2 — Dependencies Install Karo

```
npm install express socket.io

# express = HTTP server banana ke liye

# socket.io = real-time connection ke liye
```

■ *Express aur Socket.io dono ko milake use karte hain kyunki Socket.io ko ek HTTP server chahiye hota hai starting point ke liye — WebSocket connection usi HTTP server ke upar upgrade hota hai.*

Step 3 — server.js Banao (Step by Step)

Ab server.js file banao. Har line explain karunga:

```
// Line 1-3: Zaruri cheezein import karo

import express from 'express';

import { createServer } from 'http';

import { Server } from 'socket.io';
```

■ *'express' import kiya — ye ek popular Node.js framework hai jo HTTP server banana asaan banata hai.*

■ *'createServer' import kiya http module se — ye ek raw HTTP server banata hai.*

■ *'Server' import kiya socket.io se — ye Socket.io ka main class hai.*

```
// Line 4-5: App aur Server banao

const app = express();

const server = createServer(app);
```

■ *app = Express application. Ye tumhare HTTP routes handle karta hai (jaise GET /, POST /login etc.).*

■ *server = Express app ke upar ek HTTP server wrap kiya. Ye isliye zaroori hai kyunki Socket.io directly is HTTP server se attach hota hai.*

```
// Line 6-11: Socket.io Server banao

const io = new Server(server, {

cors: {
```

```
origin: 'http://localhost:5173', // frontend ka URL
methods: ['GET', 'POST']
}
});
```

■ *'new Server(server, options)' — Socket.io ko HTTP server se attach karo.*

■ *cors.origin = tumhara frontend URL. Ye security ke liye hai — sirf is URL se connections allow honge. Baad mein production mein apna real URL daalna.*

■ *cors.methods = kaun kaun se HTTP methods allowed hain handshake ke liye.*

```
// Line 12-18: Connection handle karo
io.on('connection', (socket) => {
// Ye tab chalega jab koi bhi client connect kare
console.log('Koi connect hua! ID:', socket.id);

socket.on('disconnect', () => {
console.log('Koi disconnect hua:', socket.id);
});
});
```

■ *io.on('connection', ...) — ye Socket.io ka sabse important listener hai. Jab bhi koi naya client connect hoga, ye callback function chalega.*

■ *'socket' parameter = us specific client ka object. Har connected client ka apna alag 'socket' hota hai.*

■ *socket.id = ek unique string jo Socket.io automatically deta hai har client ko — jaise ek temporary ID.*

■ *socket.on('disconnect', ...) = jab wo client disconnect ho jaaye tab kya karna hai.*

```
// Line 19: Server start karo
server.listen(3000, () => {
console.log('Server chal raha hai port 3000 pe!');
});
```

■ *server.listen(3000) — server port 3000 pe sun ne lagta hai. Ab http://localhost:3000 pe jaoge to server milega.*

Step 4 — package.json mein type:module Add Karo

ES modules (import/export) use karne ke liye package.json mein ye add karo:

```
// package.json
{
"type": "module",
"scripts": {
"start": "node server.js"
}
}
```

■ *'type: module' bolta hai Node.js ko ki import/export syntax use karo. Without this, tumhe require() use karna padega.*

Step 5 — Client Side Setup

React/Vite project mein:

```
npm install socket.io-client

# ye client-side Socket.io library hai

# server pe 'socket.io' install tha, client pe 'socket.io-client'
```

■ *Server package aur client package alag hain. Server pe 'socket.io', client pe 'socket.io-client'. Dono ka API same hai basically.*

Step 6 — Basic Client Code

```
// client.js ya App.jsx mein

import { io } from 'socket.io-client';

// Server se connect karo

const socket = io('http://localhost:3000');

// Jab connection ho jaaye

socket.on('connect', () => {

console.log('Server se connect ho gaya! Mera ID:', socket.id);

});

// Jab disconnect ho

socket.on('disconnect', () => {

console.log('Server se disconnect ho gaya');

});
```

■ *io('http://localhost:3000') — server ka URL pass karo. Socket.io automatically connect karne ki koshish karega.*

■ *socket.on('connect', ...) — ye tab chalega jab connection successfully ban jaaye.*

■ *socket.id — server pe jo ID assign hui hai wo yahan milti hai. NOTE: ye 'connect' event ke baad available hota hai, pehle nahi.*

■ *socket.id ko React ke initial render pe use mat karo — pehle connect event ka wait karo warna undefined milega.*

CHAPTER 3

Emit aur On — The Core of Socket.io

Event System Kya Hai?

Socket.io pure events pe kaam karta hai. Socho jaise **radio station**:

- emit = broadcasting karna (kuch bhejno)
- on = sun'na (kuch receive karna)

Dono side — server aur client — dono emit kar sakte hain aur dono sun sakte hain. Ye **bidirectional** hai.

■ *Jaise WhatsApp mein tum message bhej sakte ho (emit) aur receive kar sakte ho (on). Dono side same cheez kar sakte hain.*

socket.emit — Kuch Bhejne Ka Tarika

```
socket.emit(eventName, data)
// ↑ ↑
// event ka naam jo data bhejni hai
```

■ *eventName — ye tumhara custom naam hai, koi bhi rakh sakte ho. Jaise 'sendMessage', 'playerMoved', 'userTyping'. Bas dono side pe same naam hona chahiye.*

■ *data — ye wo cheez hai jo bhejni hai. Ye kuch bhi ho sakta hai: string, number, object, array, etc.*

Example — Client Se Server Ko Message Bhejo

```
// ■■ CLIENT SIDE ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
// Button click pe message bhejo
sendButton.addEventListener('click', () => {
socket.emit('sendMessage', {
text: 'Hello server!',
username: 'Anand'
});
// 'sendMessage' event naam hai
// { text, username } data hai
});
```

■ *Ye client ne ek 'sendMessage' event emit kiya with kuch data. Ab server ko sunna hoga is event ko.*

```
// ■■ SERVER SIDE ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
io.on('connection', (socket) => {

// Client ka 'sendMessage' event suno
socket.on('sendMessage', (data) => {
// data wo hi hai jo client ne bheja tha
console.log(data.text); // 'Hello server!'
```

```
console.log(data.username); // 'Anand'
});

});
```

■ *socket.on('sendMessage', ...) — server sun raha hai 'sendMessage' event ke liye. Jab bhi client ye event bhejega, ye callback chalega.*

■ *data parameter mein wo hi object aayega jo client ne emit karte waqt diya tha.*

Server Se Client Ko Bhejne Ke 3 Tarike

Tarika 1 — Sirf Us Ek Client Ko

```
// Sirf us specific socket ko bhejo jisne yeh event bheja
socket.emit('response', { msg: 'Sirf tujhe bol raha hoon' });
// socket.id wale client ko hi jaayega
```

■ *socket.emit — us SPECIFIC client ko bhejta hai jiska ye socket object hai. Baaki kisi ko nahi.*

Tarika 2 — Sabko Bhejo

```
// Sabhi connected clients ko bhejo
io.emit('announcement', { msg: 'Sabko bol raha hoon' });
// io.emit = sabko broadcast
```

■ *io.emit — is server se connected SABHI clients ko bhejta hai. Jaise ek announcement.*

Tarika 3 — Sender Ke Alaawa Sabko

```
// Bhejne wale ke alaawa baaki sab ko
socket.broadcast.emit('userJoined', { id: socket.id });
// sender ko nahi jaayega, baaki sab ko jaayega
```

■ *socket.broadcast.emit — sender ko CHHODKAR baaki sab ko bhejta hai. Ye tab useful hai jab koi join kare aur tumhe sirf baaki logo ko batana ho.*

Complete Example — Client aur Server Baat Karte Hain

```
// ■■ SERVER ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
io.on('connection', (socket) => {

// Jab client join kare, usse welcome bhejo
socket.emit('welcome', {
message: 'Server pe swagat hai!',
yourId: socket.id
});

// Baaki sabko batao naya banda aaya
socket.broadcast.emit('newUser', {
id: socket.id,
message: 'Ek naya user aaya!'
```

```
});

// Client ka message suno
socket.on('chatMessage', (data) => {
console.log('Message aaya:', data.text);
// Sabko bhej do ye message
io.emit('chatMessage', data);
});

});
```

```
// ■■ CLIENT ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
socket.on('welcome', (data) => {
console.log(data.message); // 'Server pe swagat hai!'
console.log('Mera ID:', data.yourId);
});

socket.on('newUser', (data) => {
console.log(data.message); // 'Ek naya user aaya!'
});

socket.on('chatMessage', (data) => {
displayMessage(data.text); // UI pe dikhao
});

// Message bhejo
function sendMessage(text) {
socket.emit('chatMessage', { text, from: 'Anand' });
}
```

■ *Dekho kaise flow hai: Client connect hua → server ne welcome bheja (socket.emit) → server ne baaki sab ko bataya (socket.broadcast.emit) → ab client message bhejta hai → server sabko forward karta hai (io.emit).*

CHAPTER 4

Acknowledgements — Confirm Karo Ki Message Mila

Acknowledgement Kya Hota Hai?

Normal emit mein tum bhejte ho — aur bas. Tumhe pata nahi chalta ki server ne receive kiya ya nahi, process kiya ya nahi. **Acknowledgement** ek mechanism hai jisme sender ko ek callback milti hai jab receiver ne event process kar liya.

> ■ *Jaise tum kisi ko WhatsApp pe bhejo aur aapko double tick mile — confirmation ki message mila. Acknowledgement wahi hai code mein.*

Acknowledgement Kaise Kaam Karta Hai?

Emit karte waqt last argument mein ek **function (callback)** dete hain. Server us function ko call karta hai apna jawab dene ke liye.

Example — Room Create Karo with Confirmation

```
// ■■ CLIENT ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
socket.emit('createRoom', { name: 'gaming-room' }, (response) => {
// ↑
// ye callback hai — server isse call karega

if (response.success) {
console.log('Room ban gayi! ID:', response.roomId);
} else {
console.log('Error:', response.error);
}
});
```

> ■ *emit ka teesra argument ek function hai. Ye callback tab chalegi jab server isse call karega. response mein wo data hoga jo server ne diya.*

```
// ■■ SERVER ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
socket.on('createRoom', (data, callback) => {
// ↑
// callback last parameter mein milti hai

if (!data.name) {
// Error bhejo callback ke through
return callback({ success: false, error: 'Name zaroori hai' });
}

const roomId = Math.random().toString(36).slice(2, 8);
```

```
rooms.set(roomId, { name: data.name });

// Success confirm karo
callback({ success: true, roomId });
// ↑
// Ye client ki callback function ko call karega
});
```

■ *Server ke socket.on mein last parameter callback function hoti hai. Server jab callback(data) call karta hai, client ke paas wo data pahunch jaata hai.*

■ *Ye normal function call jaisa lagta hai but actually network ke through hota hai — Socket.io ye magic karta hai.*

Acknowledgement with Error Handling

```
// Client side — proper error handling ke saath
socket.emit('loginUser', { username, password }, (res) => {
if (res.status === 'ok') {
console.log('Login ho gaya!', res.user);
redirectToDashboard();
} else if (res.status === 'wrong_password') {
showError('Password galat hai');
} else if (res.status === 'user_not_found') {
showError('User nahi mila');
}
});
```

■ *Acknowledgements se tum request-response pattern create kar sakte ho Socket.io mein — bilkul REST API jaisa but real-time connection ke saath.*

✦ *Jab bhi tumhe server se confirmation chahiye — room join hua ya nahi, message save hua ya nahi, login successful ya nahi — Acknowledgement use karo.*

CHAPTER 5

Namespaces — Alag Channels Banao

Namespace Kya Hota Hai?

Socho tumhara ek bada ghar hai. Ghar mein alag alag kamre hain — drawing room, bedroom, kitchen. Har kamre mein alag log hain, alag baatein hoti hain. **Namespaces waisi hi alag alag jagahein hain Socket.io connection ke andar.**

Ek hi physical connection (WebSocket) ke upar multiple logical channels bana sakte ho. Har namespace ke apne:

- Events alag hain
- Rooms alag hain
- Middleware alag hai
- Connected clients alag hain

■ *Example: Ek app hai jisme chat bhi hai aur notifications bhi. Chat ke liye /chat namespace, notifications ke liye /notifications namespace. Dono alag chalne wale real-time channels.*

Default Namespace — /

Jab tum simple io.on('connection') karte ho — wo **default namespace /** use karta hai. Ye automatically milta hai.

```
// Ye dono same hain:
io.on('connection', handler);
io.of('/').on('connection', handler);
// dono default namespace '/' pe kaam karte hain
```

Custom Namespace Banao

```
// ■■ SERVER ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■

// /chat namespace banao
const chatNS = io.of('/chat');
// ↑
// namespace ka naam — koi bhi rakh sakte ho

chatNS.on('connection', (socket) => {
console.log('Koi /chat mein connect hua:', socket.id);

socket.on('message', (text) => {
// Sirf /chat namespace ke clients ko bhejo
chatNS.emit('message', { text, from: socket.id });
});
});
```

```
// /admin namespace banao
const adminNS = io.of('/admin');

adminNS.on('connection', (socket) => {
console.log('Admin connect hua:', socket.id);
// /admin ka apna alag duniya hai
});
```

```
// ■■ CLIENT ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■

// /chat namespace se connect karo
const chatSocket = io('http://localhost:3000/chat');
// ↑
// URL ke end mein namespace

// /admin namespace se connect karo (alag socket!)
const adminSocket = io('http://localhost:3000/admin');

// Dono alag connections hain, ek doosre ke events nahi sunenge
chatSocket.on('message', (data) => {
displayChatMessage(data);
});
```

■ Notice karo — client ke paas alag alag socket objects hain (/chat ke liye chatSocket, /admin ke liye adminSocket). Ye physically ek connection use karte hain but logically alag hain.

Namespace Middleware — Auth Karo Connection Se Pehle

```
// Sirf /admin namespace pe auth check karo
adminNS.use((socket, next) => {
// ↑ ↑
// socket next function — agar sab theek hai to next() bulao

const token = socket.handshake.auth.token;
// socket.handshake mein connection ke waqt ki saari info hoti hai

if (!token) {
return next(new Error('Token nahi diya!'));
// next(error) = connection reject kar do
}

const user = verifyJWT(token);
if (!user.isAdmin) {
return next(new Error('Admin access nahi hai'));
```

```
}

socket.user = user; // user ko socket pe attach karo

next(); // sab theek — connection allow karo

});
```

■ *Middleware connection se PEHLE chalta hai. Agar next(error) bulaya to client ko error milega aur connection nahi banegi. Agar next() bulaya to connection ban jaati hai.*

■ *socket.handshake — ye object connection ke time ki saari info rakhta hai: token, query params, IP address, headers, etc.*

CHAPTER 6

Rooms — Groups Banao

Room Kya Hota Hai?

Rooms ek **server-side concept** hai — clients ko groups mein organize karne ka tarika. Ek socket multiple rooms mein ho sakta hai. Ek room mein multiple sockets ho sakte hain.

■ *Socho WhatsApp groups jaisa. Ek banda multiple groups mein ho sakta hai. Ek group mein message aaye to sirf group members ko dikhe. Rooms bilkul waise kaam karte hain.*

❖ *Important: Rooms SERVER-SIDE sirf hote hain. Client ko pata nahi hota ki kaun kaun se rooms hain. Sab kuch server handle karta hai.*

Room Join Karna

```
io.on('connection', (socket) => {

socket.on('joinRoom', (roomId) => {

socket.join(roomId);

// ↑

// is socket ko is room mein daal do

console.log(`${socket.id} ne room join kiya: ${roomId}`);

console.log('Is socket ke rooms:', socket.rooms);

// Set { 'socket-id-xyz', 'roomId' }

// Note: socket apna khud ka ID bhi ek room hai

});

});
```

■ *socket.join(roomId) — is socket ko us room mein daal do. Ek socket kitne bhi rooms mein join ho sakta hai.*

■ *socket.rooms — ye ek Set hai jisme us socket ke saare rooms ke naam hain. Har socket apne ID wale room mein automatically hota hai.*

Room Mein Message Bhejne Ke Tarike

```
// Tarika 1: Room ke SABHI members ko bhejo (sender bhi)

io.to(roomId).emit('roomMessage', data);

// ↑

// io.to = us room ke sab clients

// Tarika 2: Room ke sab members ko bhejo SENDER CHHOD KE

socket.to(roomId).emit('roomMessage', data);

// ↑

// socket.to = sender ke alaawa room ke sab clients
```

```
// Tarika 3: Multiple rooms ko ek saath
io.to('room1').to('room2').emit('event', data);

// Tarika 4: Specific socket ko (private message)
io.to(targetSocketId).emit('privateMsg', data);
```

■ *io.to() aur socket.to() mein farak: io.to() mein sender bhi shamil hai, socket.to() mein sender chhut jaata hai.*

Complete Room Example — Chat App Ka Core

```
io.on('connection', (socket) => {

// User room join kare
socket.on('joinRoom', ({ roomId, username }) => {
socket.join(roomId);
socket.data.username = username;
// socket.data — is socket pe custom data store karo

// Room ke baaki log ko batao naya banda aaya
socket.to(roomId).emit('userJoined', {
username,
message: `${username} room mein aaya!`
});
});

// Message bhejo room mein
socket.on('sendMessage', ({ roomId, text }) => {
io.to(roomId).emit('newMessage', {
from: socket.data.username,
text,
time: new Date().toLocaleTimeString()
});
// io.to() use kiya taaki sender ko bhi apna message dikhe
});

// Typing indicator
socket.on('typing', ({ roomId, isTyping }) => {
socket.to(roomId).emit('userTyping', {
username: socket.data.username,
isTyping
});
```

```
// socket.to() use kiya — sender ko nahi dikhna apna typing
});

// Room leave karna
socket.on('leaveRoom', (roomId) => {
socket.leave(roomId);
socket.to(roomId).emit('userLeft', {
username: socket.data.username
});
});

});
```

Room Introspection — Room ke Baare Mein Pata Karo

```
// Room mein kitne/kaun log hain
const sockets = await io.in(roomId).fetchSockets();
console.log('Room mein log:', sockets.length);

// Is socket ke kaun kaun se rooms hain
console.log(socket.rooms);
// Output: Set { 'socket-id-xyz', 'gaming-room', 'team-5' }

// Check karo socket is room mein hai ya nahi
const isInRoom = socket.rooms.has('gaming-room');
console.log(isInRoom); // true ya false
```

■ *fetchSockets() ek async function hai, isliye await use karo. Ye us room ke sabhi connected sockets ka array deta hai.*

Private Message — Seedha Kisi Ko DM Karo

Yaad hai maine kaha tha ki har socket apne ID ke naam wale room mein automatically hota hai? Usi se private messages kaam karte hain:

```
socket.on('privateMessage', ({ toSocketId, text }) => {
// targetSocketId ka room = targetSocketId hi hai
io.to(toSocketId).emit('privateMessage', {
from: socket.id,
text,
time: Date.now()
});
});
```

```
// Client side se use karo:
socket.emit('privateMessage', {
toSocketId: 'abc123xyz', // us user ka socket.id
text: 'Arre yaar kya haal hai'
});
```

■ *io.to(socketId) matlab us socket ke naam wale room mein emit karo. Since har socket apne ID wale room mein akela hota hai, message sirf usi tak jaata hai.*

CHAPTER 7

Middleware aur Authentication

Middleware Kya Hota Hai?

Middleware ek function hai jo connection establish hone SE PEHLE chalta hai. Socho isko ek **security guard** jaisa — jo check karta hai 'ID card hai kya?' aur phir andar jaane deta hai ya rok deta hai.

■ *Jaise hotel mein bina ID dikhaaye room nahi milta, waise hi bina valid token ke Socket.io connection nahi banegi — agar tumne middleware lagaya hai to.*

JWT Authentication — Step by Step

JWT (JSON Web Token) ek popular authentication method hai. Jab user login karta hai to server ek token deta hai. Har request/connection mein wo token bhejte hain proof ke liye.

Step 1 — JWT install karo

```
npm install jsonwebtoken
```

Step 2 — Server pe middleware lagao

```
import jwt from 'jsonwebtoken';

const JWT_SECRET = 'ye_secret_key_bahut_lamba_rakho_production_mein';

// io.use() = global middleware — har connection ke liye chalega
io.use((socket, next) => {
// ↑ ↑
// socket next() = allow karo, next(err) = reject karo

// Token ko do jagah se lo — auth object ya query params
const token = socket.handshake.auth.token
|| socket.handshake.query.token;

// Token nahi diya to reject karo
if (!token) {
return next(new Error('Token nahi diya!'));
// next(error) = connection band kar do
}

try {
// Token verify karo — agar galat hai to error throw hoga
const decoded = jwt.verify(token, JWT_SECRET);
// decoded = { id: 1, name: 'Anand', role: 'user', iat: ..., exp: ... }

socket.user = decoded; // user data socket pe save karo
```

```
// ab socket.user har jagah available hoga

next(); // sab theek — connection allow karo
} catch (err) {
next(new Error('Token invalid ya expire ho gaya!'));
}
});

// Ab connection mein socket.user available hai
io.on('connection', (socket) => {
console.log('Connected user:', socket.user.name);
console.log('User role:', socket.user.role);
});
```

■ *socket.handshake.auth.token — client ne connection ke waqt jo auth object diya tha usmein se token lo.*

■ *jwt.verify(token, secret) — token ko verify karta hai. Agar token sahi hai to decoded data milta hai. Agar galat/expire hai to error throw hota hai.*

■ *socket.user = decoded — ab ye data is socket object ke saath hamesha available rehega jab tak connection hai.*

Step 3 — Client se Token Bhejo

```
// Login ke baad token localStorage mein save hota hai (usually)
const token = localStorage.getItem('authToken');

const socket = io('http://localhost:3000', {
auth: {
token: token
// ya shorter: token
}
// auth object handshake ke saath server ko jaata hai
});

// Agar auth fail hua to ye event fire hoga
socket.on('connect_error', (err) => {
console.log('Connection error:', err.message);
// err.message = jo bhi server ne next(new Error('...')) mein diya

if (err.message === 'Token nahi diya!') {
window.location.href = '/login'; // login pe bhejo
}
});
```

■ *io() ke options mein auth object pass karo. Ye socket.handshake.auth pe available hoga server side pe.*

Per-Event Middleware — Har Event Ke Pehle

```
// socket.use() = is specific socket ke har event ke pehle chalega

io.on('connection', (socket) => {

socket.use(([event, ...args], next) => {

// ↑ ↑

// event naam allow/reject

console.log('Event aaya:', event);

// Admin-only events check karo

if (event === 'deleteUser' && socket.user.role !== 'admin') {

return next(new Error('Admin hi delete kar sakta hai'));

}

next(); // event continue hone do

});

socket.on('deleteUser', (userId) => {

// Ye sirf tab chalega jab middleware ne allow kiya

});

});
```

■ *socket.use() — connection level pe nahi, event level pe middleware. Har event se pehle check karo.*

CHAPTER 8

Advanced Emit Patterns

Saare Emit Variants Ek Jagah

Ab tak tune basic emit seekha. Yahan pe sab tarike hain server se emit karne ke — har ek ka use case alag hai:

```
// 1. Sirf is ek socket ko
socket.emit('event', data);

// 2. Sabhi connected clients ko
io.emit('event', data);

// 3. Sender ke alaawa sabko
socket.broadcast.emit('event', data);

// 4. Ek room ke sab logon ko (sender bhi)
io.to('roomName').emit('event', data);

// 5. Room ko — lekin sender ko nahi
socket.to('roomName').emit('event', data);

// 6. Kisi bhi specific socket ko (uska ID use karke)
io.to(targetSocketId).emit('event', data);

// 7. Multiple rooms ko ek saath
io.to('room1').to('room2').emit('event', data);
```

volatile.emit — Unreliable Data Ke Liye

```
// Agar client ready nahi hai to event skip kar do
socket.volatile.emit('liveLocation', {
lat: 25.5941,
lng: 85.1376
});
```

■ *volatile.emit — agar us waqt client ka buffer full hai ya ready nahi hai, to ye event drop ho jaata hai. Yani data lose ho sakta hai.*

■ *Use case: Live GPS location, live score updates, mouse position — jahan missed ek update okay hai kyunki agla update aane wala hai.*

■ *Normal emit ke liye use mat karo jahan data important ho (messages, orders, etc.).*

compress — Compression Control

```
// Compression ke saath (default behavior)
```

```
socket.emit('bigData', largeObject);

// Compression band karo
socket.compress(false).emit('smallData', { x: 1 });
// Choti data ke liye compression off karna faster hota hai
// kyunki compress/decompress ka overhead nahi hoga
```

■ *By default Socket.io data compress karta hai. Choti objects ke liye compression overhead zyada hoti hai to .compress(false) use karo.*

timeout — Acknowledgement Ka Wait Time

```
// 5 seconds mein agar client respond na kare to error
socket.timeout(5000).emit('importantEvent', data, (err, response) => {
if (err) {
// Client ne 5 seconds mein respond nahi kiya
console.log('Timeout ho gaya! Client ne jawab nahi diya.');
retryOrAlert();
} else {
console.log('Response mila:', response);
}
});
```

■ *Normal acknowledgement mein agar client kabhi respond na kare to callback hamesha wait karti rehti hai. .timeout(ms) se ek time limit set hoti hai.*

Express Route Se Emit — Server Se Event Trigger Karo

```
// HTTP request aaye aur sabko notify karo
app.post('/new-order', express.json(), (req, res) => {
const order = req.body;

// Database mein save karo (example)
saveOrder(order);

// Sabhi connected clients ko batao
io.emit('newOrder', {
orderId: order.id,
item: order.item,
message: 'Naya order aaya!'
});

res.json({ success: true });
});
```

■ *io object ko Express routes mein bhi use kar sakte ho. Ye useful hai jab koi REST API call ho aur tumhe real-time clients ko bhi notify karna ho.*

CHAPTER 9

Socket.io with React

Sabse Badi Galti Jo Log Karte Hain

Ye bilkul galat hai — kabhi mat karo:

```
// GALAT! Component ke andar socket banao mat!
function ChatComponent() {
const socket = io('http://localhost:3000');
// Har re-render pe naya socket connection banega!
// 10 re-renders = 10 connections = server overload!
}
```

■ *Component ke andar io() call karne se har render pe ek naya WebSocket connection khulta hai. 10 renders = 10 connections. Ye bahut badi problem hai.*

Sahi Tarika — Singleton Pattern

Socket ek baar banao, bahar, aur sab jagah import karo:

```
// src/socket.js — YE FILE EKBAR BANTI HAI
import { io } from 'socket.io-client';

// Socket COMPONENT KE BAHAR banao
// Ye ek baar execute hota hai — ek hi connection
export const socket = io('http://localhost:3000', {
autoConnect: false,
// autoConnect: false matlab abhi connect mat karo
// manually connect karenge login ke baad

auth: {
token: localStorage.getItem('token') // ya '' agar nahi hai
},

reconnection: true,
reconnectionAttempts: 5,
reconnectionDelay: 1000,
});
```

■ *autoConnect: false rakho agar pehle login chahiye. Login ke baad manually socket.connect() bulao.*

■ *Is file ko import karo jahan bhi chahiye — same socket object milega, naya nahi banega.*

Custom useSocket Hook — Event Sunne Ka Sahi Tarika

```
// src/hooks/useSocket.js
import { useEffect, useCallback } from 'react';
import { socket } from '../socket';

export function useSocket(eventName, handler) {
// useCallback se handler ko stable banao
// warna har render pe naya function banta hai aur
// useEffect baar baar chalega
const stableHandler = useCallback(handler, []);

useEffect(() => {
// Event listener add karo
socket.on(eventName, stableHandler);

// CLEANUP — jab component unmount ho to listener hatao
return () => {
socket.off(eventName, stableHandler);
// off nahi kiya to MEMORY LEAK hogi
// Aur event baar baar fire hoga
};
}, [eventName, stableHandler]);
}

// KAISE USE KARO:
// Component mein:
useSocket('newMessage', (msg) => {
setMessages(prev => [...prev, msg]);
});
```

■ *useCallback handler ko memoize karta hai — matlab ek hi function reference rehta hai renders ke beech, useEffect baar baar nahi chalta.*

■ *return () => socket.off() — ye cleanup function hai. Component unmount hone pe listener hat jaata hai. HAMESHA karo ye, warna same event ke 10 listeners ban jaate hain aur ek event 10 baar fire hota hai.*

Login/Logout Pattern

```
import { socket } from '../socket';

// Login ke baad connect karo
async function handleLogin(credentials) {
const response = await fetch('/api/login', {
method: 'POST',
```

```
body: JSON.stringify(credentials)
});
const { token } = await response.json();

localStorage.setItem('token', token);

// Socket ka auth update karo
socket.auth = { token };

// Ab connect karo
socket.connect();
}

// Logout pe disconnect karo
function handleLogout() {
socket.disconnect();
localStorage.removeItem('token');
}
```

■ *Login ke baad socket.auth update karo aur socket.connect() bulao. Logout pe socket.disconnect() zaroor bulao warna background mein connection open rehta hai.*

CHAPTER 10

Scaling with Redis Adapter

Problem Kya Hai?

Jab sirf ek Node.js process chal raha hota hai, sab theek hai. Lekin production mein multiple processes chalate hain (speed aur reliability ke liye). Tab problem hoti hai:

- Process 1 mein User A connected hai
- Process 2 mein User B connected hai
- User A ne io.emit() kiya — sirf Process 1 ke clients ko jaayega
- User B ko nahi milega — wo Process 2 pe hai!

■ *Socho ek hi WhatsApp ke 2 alag servers hain. Server 1 pe log message bheje to sirf server 1 ke users dekhe, server 2 ke nahi. Ye toh bekaar system hai.*

Solution — Redis Adapter

Redis Adapter sabhi processes ko ek message bus se connect karta hai. Koi bhi process emit kare — Redis ke through sabhi processes ko milta hai.

```
npm install @socket.io/redis-adapter redis
```

```
import { createAdapter } from '@socket.io/redis-adapter';
import { createClient } from 'redis';

// Ek publisher client banao
const pubClient = createClient({ url: 'redis://localhost:6379' });

// Ek subscriber client banao (publisher ka duplicate)
const subClient = pubClient.duplicate();

// Dono ko connect karo
await Promise.all([pubClient.connect(), subClient.connect()]);

// Socket.io ko batao Redis use karo
io.adapter(createAdapter(pubClient, subClient));

// Ab sabhi processes mein io.emit() kaam karega!
console.log('Redis adapter active hai');
```

■ *pubClient = messages bhejne ke liye Redis connection. subClient = messages sunne ke liye (same Redis, alag connection). Ye pub/sub pattern hai.*

■ *io.adapter(createAdapter(...)) ke baad tumhara existing code bilkul same kaam karta hai — difference sirf ye hai ki ab messages cross-process jaate hain.*

Sticky Sessions — Load Balancer Config

Socket.io pehle HTTP polling karta hai, phir WebSocket pe upgrade hota hai. Is process mein ek client ke saare requests SAME server pe jaane chahiye.

■ *Agar load balancer alag alag requests alag servers pe bhejta hai, to WebSocket handshake fail hoga. Isliye 'Sticky Sessions' zaroori hai.*

- Nginx: upstream block mein ip_hash add karo
- AWS ALB: target group mein sticky sessions enable karo
- PM2: cluster mode mein --sticky-sessions flag use karo

CHAPTER 11

Error Handling aur Reconnection

Client Reconnection — Options Samjho

Network chali jaaye, server restart ho jaaye — Socket.io automatically reconnect karne ki koshish karta hai. Ye options se control kar sakte ho:

```
const socket = io('http://localhost:3000', {

reconnection: true,

// true = automatic reconnect karo (default bhi yahi hai)

reconnectionAttempts: 5,

// 5 baar try karo, phir give up karo

// Infinity rakho agar hamesha try karna ho

reconnectionDelay: 1000,

// Pehli retry 1 second baad

reconnectionDelayMax: 10000,

// Max 10 seconds tak wait karo retries ke beech

// (har retry mein delay badhta jaata hai — exponential backoff)

randomizationFactor: 0.5,

// Thodi random variation add karo timing mein

// Taaki sab clients ek saath reconnect na karein (server overload)

timeout: 20000,

// Pehle connection ke liye 20 seconds wait karo

});
```

Saare Lifecycle Events

```
// Jab connect ho

socket.on('connect', () => {

console.log('Connected! ID:', socket.id);

hideOfflineBar();

});

// Jab disconnect ho

socket.on('disconnect', (reason) => {
```

```
console.log('Disconnect hua. Reason:', reason);
showOfflineBar('Connection gayi, reconnect ho raha hai...');
});

// Jab connect karne mein error aaye
socket.on('connect_error', (err) => {
console.log('Connection error:', err.message);
});

// Jab successfully reconnect ho
socket.on('reconnect', (attemptNumber) => {
console.log('Reconnect hua! Attempt number:', attemptNumber);
hideOfflineBar();
fetchMissedMessages(); // jo messages miss hue wo lo
});

// Jab reconnect karne ki koshish ho
socket.on('reconnect_attempt', (n) => {
console.log('Retry attempt number:', n);
});

// Jab saari retries fail ho jaayein
socket.on('reconnect_failed', () => {
console.log('Sab retries fail ho gayi!');
showError('Server se connection nahi bana. Refresh karo.');
});
```

■ *Ye events use karke UI mein 'You are offline' bar dikha sakte ho, aur reconnect hone pe automatically missed data fetch kar sakte ho.*

Disconnect Reasons — Kyon Gaya Connection?

Reason	Matlab
io server disconnect	Server ne force disconnect kiya (socket.disconnect() call kiya)
io client disconnect	Client ne khud disconnect kiya (socket.disconnect() call kiya)
ping timeout	Server ne ping ka jawab nahi diya — server down ya network issue
transport close	Connection close hua — tab band ki, network gayi
transport error	Connection mein error aaya

Graceful Server Shutdown

```
// Jab server band karo to clients ko batao
process.on('SIGTERM', () => {
// Pehle clients ko warn karo
io.emit('serverRestart', {
message: 'Server 5 second mein restart ho raha hai',
retryIn: 5000
});

// Phir sabke connections close karo
io.close(() => {
server.close(() => {
console.log('Server gracefully band ho gaya');
process.exit(0);
});
});
});
```

CHAPTER 12

Security Best Practices

1. CORS Sahi Se Lagao

CORS (Cross-Origin Resource Sharing) control karta hai ki kaun kaun se websites tumhare Socket.io server se connect kar sakti hain.

```
// GALAT — Sab allow karo (Production mein kabhi mat karo!)
const io = new Server(server, {
cors: { origin: '*' }
// '*' = koi bhi website connect kar sakti hai = security hole!
});

// SAHI — Specific URLs allow karo
const io = new Server(server, {
cors: {
origin: [
'https://tumhara-app.com',
'https://staging.tumhara-app.com',
'http://localhost:5173' // development ke liye
],
methods: ['GET', 'POST'],
credentials: true // cookies bhejni hain to true karo
}
});
```

2. Rate Limiting — Spam Se Bachao

```
npm install rate-limiter-flexible
```

```
import { RateLimiterMemory } from 'rate-limiter-flexible';

// Har IP ke liye: 1 second mein max 10 events
const limiter = new RateLimiterMemory({
points: 10, // 10 points (events)
duration: 1, // 1 second window
});

io.on('connection', (socket) => {

// socket.use() = har event se pehle ye chalega
```

```
socket.use(async ([event, ...args], next) => {
try {
// Is IP ka ek point consume karo
await limiter.consume(socket.handshake.address);
next(); // limit ke andar hai — allow karo
} catch {
// Limit cross ho gayi
next(new Error('Bahut zyada requests! Thoda ruko.'));
}
});

});
```

3. Input Validation — Kabhi Client Ka Data Trust Mat Karo

```
// Har event mein data validate karo
socket.on('sendMessage', (data) => {

// Basic checks
if (!data || typeof data !== 'object') {
return socket.emit('error', { msg: 'Invalid data format' });
}

if (!data.text || typeof data.text !== 'string') {
return socket.emit('error', { msg: 'Text field missing' });
}

if (data.text.length > 500) {
return socket.emit('error', { msg: 'Message bahut lamba hai (max 500)' });
}

// XSS se bachao — HTML characters escape karo
const safeText = data.text
.replace(/&/g, '&amp;')
.replace(/</g, '&lt;')
.replace(/>/g, '&gt;');

io.emit('newMessage', { text: safeText, from: socket.user.name });
});
```

Security Checklist

- HTTPS/WSS use karo production mein — HTTP/WS kabhi nahi

- CORS mein specific origins likho, * mat likho
- JWT ya session se har connection authenticate karo
- Har event mein incoming data validate karo
- Rate limiting lagao spam aur DDoS se bachne ke liye
- socket.id ya internal data kabhi doosre users ko mat dikhaao
- User input ko HTML se escape karo (XSS prevention)

CHAPTER 13

Testing Socket.io

Setup

```
npm install --save-dev jest

# socket.io-client pehle se install hai

# package.json mein: "test": "jest"
```

Basic Test — Ping Pong

```
// server.test.js

import { createServer } from 'http';

import { Server } from 'socket.io';

import { io as Client } from 'socket.io-client';

let io, server, clientSocket;

// Test se pehle server start karo

beforeAll((done) => {

server = createServer();

io = new Server(server);

server.listen(() => {

// Random free port pe listen karo

const { port } = server.address();

clientSocket = Client('http://localhost:' + port);

// Server side event setup

io.on('connection', (socket) => {

socket.on('ping', (callback) => {

callback('pong'); // acknowledgement bhejo

});

});

clientSocket.on('connect', done);

// done() bulao jab connect ho jaaye — test start ho

});

});

// Sab tests ke baad cleanup
```

```
afterAll(() => {
io.close();
clientSocket.close();
server.close();
});

// Actual test
test('ping bhejo to pong milna chahiye', (done) => {
clientSocket.emit('ping', (response) => {
expect(response).toBe('pong');
done(); // async test complete
});
});
```

■ *done() function Jest ko batata hai ki ye async test complete ho gaya. Bina done() ke test timeout ho jaata hai.*

Multiple Clients Test — Broadcast Check Karo

```
test('ek client bheje to doosre ko milna chahiye', (done) => {
const { port } = server.address();
const client2 = Client('http://localhost:' + port);

client2.on('connect', () => {
// Client2 sunne lage
client2.on('broadcast', (msg) => {
expect(msg).toBe('Hello sab!');
client2.close(); // cleanup
done();
});

// Client1 bheje
clientSocket.emit('broadcast', 'Hello sab!');
});
});

// Server side mein ye hona chahiye:
// socket.on('broadcast', (msg) => {
// socket.broadcast.emit('broadcast', msg);
// });
```

CHAPTER 14

Real Project — Pura Chat App

Sab cheezein milake ek production-ready chat app banate hain — JWT auth, rooms, typing indicator, online users.

server.js — Complete Code

```
import express from 'express';
import { createServer } from 'http';
import { Server } from 'socket.io';
import jwt from 'jsonwebtoken';

const app = express();
const server = createServer(app);
const io = new Server(server, {
cors: { origin: 'http://localhost:5173' }
});

const JWT_SECRET = 'super_secret_key_123';

// Sab online users track karne ke liye
// Map: socketId -> { name, socketId }
const onlineUsers = new Map();

// Step 1: Auth middleware — connection se pehle
io.use((socket, next) => {
const token = socket.handshake.auth.token;
if (!token) return next(new Error('Login karo pehle'));
try {
socket.user = jwt.verify(token, JWT_SECRET);
next();
} catch {
next(new Error('Token invalid hai'));
}
});

io.on('connection', (socket) => {

// User online list mein add karo
onlineUsers.set(socket.id, {
name: socket.user.name,
```

```
socketId: socket.id
});

// Sabko updated online list bhejo
io.emit('onlineUsers', [...onlineUsers.values()]);

// Room join karna
socket.on('joinRoom', (roomId) => {
socket.join(roomId);
// Room mein baaki sabko batao
socket.to(roomId).emit('notice',
`${socket.user.name} room mein aa gaya!`
);
});

// Message bhejana
socket.on('message', ({ roomId, text }) => {
if (!text || text.trim() === '') return;

io.to(roomId).emit('message', {
from: socket.user.name,
text: text.trim(),
time: new Date().toLocaleTimeString('hi-IN')
});
});

// Typing indicator
socket.on('typing', ({ roomId, isTyping }) => {
socket.to(roomId).emit('typing', {
name: socket.user.name,
isTyping
});
});

// Disconnect hone pe
socket.on('disconnect', () => {
onlineUsers.delete(socket.id);
io.emit('onlineUsers', [...onlineUsers.values()]);
});

});
```

```
server.listen(3000, () => console.log('Chat server: port 3000'));
```

React — useChatRoom Hook

```
// src/hooks/useChatRoom.js
import { useState, useEffect } from 'react';
import { socket } from '../socket';

export function useChatRoom(roomId) {
const [messages, setMessages] = useState([]);
const [typing, setTyping] = useState(null);
const [online, setOnline] = useState([]);

useEffect(() => {
// Room join karo
socket.emit('joinRoom', roomId);

// Events suno
socket.on('message', (msg) => setMessages(p => [...p, msg]));
socket.on('typing', (data) => {
setTyping(data.isTyping ? data.name : null);
});
socket.on('onlineUsers',(users) => setOnline(users));
socket.on('notice', (text) => {
setMessages(p => [...p, { system: true, text }]);
});

// Cleanup — jab component unmount ho ya roomId change ho
return () => {
['message','typing','onlineUsers','notice']
.forEach(ev => socket.off(ev));
};
}, [roomId]);

const sendMessage = (text) =>
socket.emit('message', { roomId, text });

const sendTyping = (isTyping) =>
socket.emit('typing', { roomId, isTyping });

return { messages, typing, online, sendMessage, sendTyping };
```

```
}
```

CHAPTER 15

Quick Reference Cheatsheet

Server — Sab Emit Tarike

```
socket.emit('ev', data) // sirf is ek client ko
io.emit('ev', data) // sab clients ko
socket.broadcast.emit('ev', data) // sender chhod ke sab ko
io.to('room').emit('ev', data) // room ke sab ko (sender bhi)
socket.to('room').emit('ev', data) // room ke sab ko (sender nahi)
io.to(socketId).emit('ev', data) // specific client ko ID se
io.to('r1').to('r2').emit('ev', data) // multiple rooms
socket.volatile.emit('ev', data) // drop karo agar ready nahi
socket.compress(false).emit('ev', data) // bina compression
socket.timeout(ms).emit('ev', d, cb) // time limit ke saath
```

Client — Key Methods

```
const socket = io(URL, { auth, reconnection, timeout, ... });
socket.connect(); // manually connect
socket.disconnect(); // disconnect karo

socket.emit('ev', data, [ackCallback]);
socket.on('ev', handler);
socket.once('ev', handler); // sirf ek baar suno
socket.off('ev', handler); // listener hatao
socket.offAny(); // sab listeners hatao
socket.onAny((ev, ...args) => {}); // sab events suno

socket.id // unique ID (connect ke baad)
socket.connected // true/false
socket.rooms // Set of rooms (server-side only)
```

Common Errors aur Fix

Error	Kyun Aata Hai	Fix
CORS error	Frontend URL allow nahi	Server mein cors.origin mein frontend URL daalo
Multiple connections React	Component andar io() call kiya	socket.js file banao, component ke bahar

Event baar baar fire hota	socket.off() nahi kiya	useEffect return mein socket.off() karo
socket.id undefined	connect event se pehle use kiya	'connect' event ke andar use karo
Auth fail hota hai	Token sahi se nahi bheja	io() mein auth: { token } pass karo
Scaling issue	Multiple servers, alag memory	Redis Adapter lagao

---

END OF DOCUMENT

Socket.io Zero to Hero — Hinglish Edition